Cloud Native
RabbitMQ

Mensajería asíncrona, Spring Boot, React/Vite y Docker

Bastián Pérez
Camila Concha
Bastián Balladares

Introducción

Implementar y verificar un sistema de mensajería asíncrona con RabbitMQ. Se trabajó progresivamente desde la validación del entorno hasta una integración completa entre React/Vite, Spring Boot y RabbitMQ, comprobando tanto el flujo normal de mensajes como escenarios de falla y recuperación.

Objetivos desarrollados

- Verificar Docker, Java y Maven como base del laboratorio.
- Levantar RabbitMQ mediante Docker Compose y acceder a RabbitMQ Management.
- Implementar una cola básica con productor y consumidor en Spring Boot.
- Configurar un DirectExchange con múltiples colas, bindings y routing keys.
- Exponer un endpoint REST para publicar mensajes desde clientes externos.
- Integrar un frontend React/Vite para enviar logs INFO, WARNING y ERROR.
- Comprobar almacenamiento temporal de mensajes cuando el consumidor está detenido.
- Validar el comportamiento de una routing key sin binding asociado.

Arquitectura
y conceptos aplicados

La solución final utiliza dos protocolos: HTTP entre el frontend y Spring Boot, y AMQP entre Spring Boot y RabbitMQ. RabbitMQ actúa como broker intermediario y desacopla la publicación del procesamiento de los mensajes

React / Vite — localhost:5173

↓ HTTP POST /log

Spring Boot — localhost:8080

↓ RabbitTemplate / AMQP

RabbitMQ — logs_direct_exchange

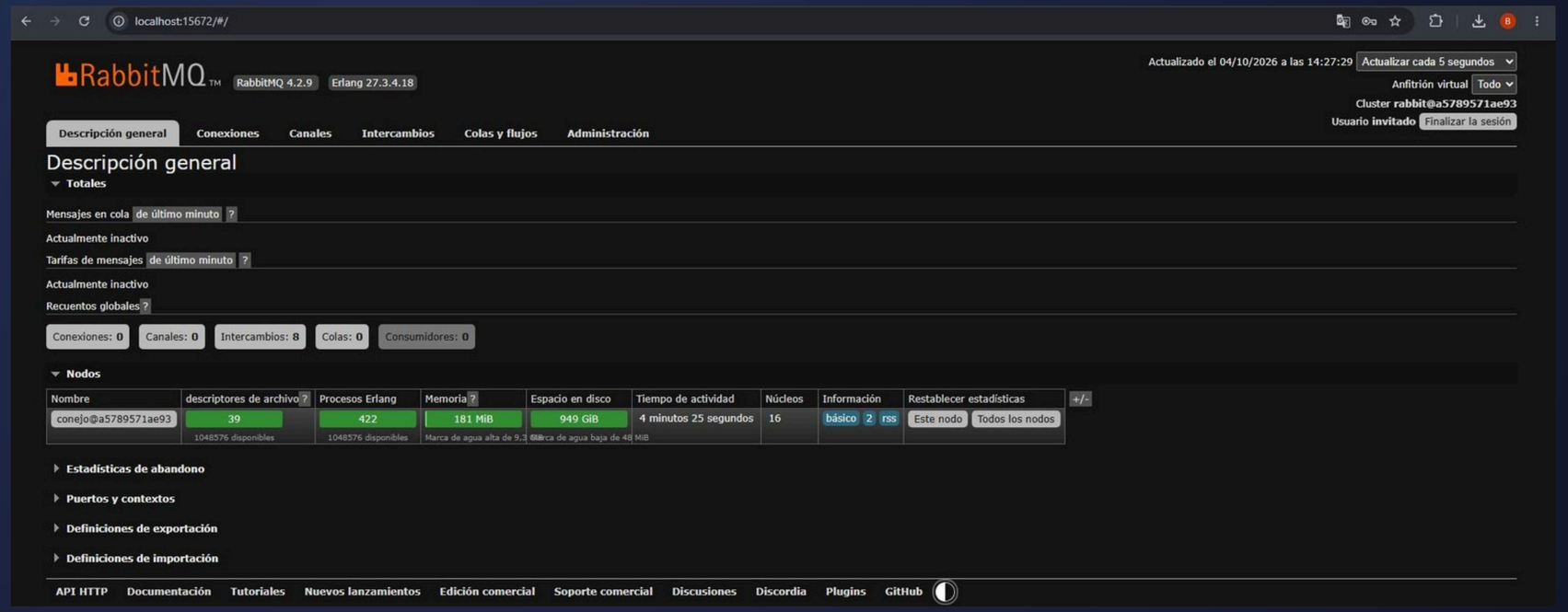
↓ Routing keys: INFO · WARNING · ERROR

all_logs_queue / errors_only_queue → consumidores

- Producer: componente que genera y publica mensajes.
- Exchange: recibe mensajes y decide a qué cola enviarlos.
- Routing key: clave utilizada por el exchange para decidir la ruta.
- Binding: relación entre un exchange y una cola asociada a una binding key.
- Queue: almacena mensajes hasta que un consumidor pueda procesarlos.
- Consumer: componente que recibe y procesa mensajes desde una cola

RabbitMQ con Docker Compose

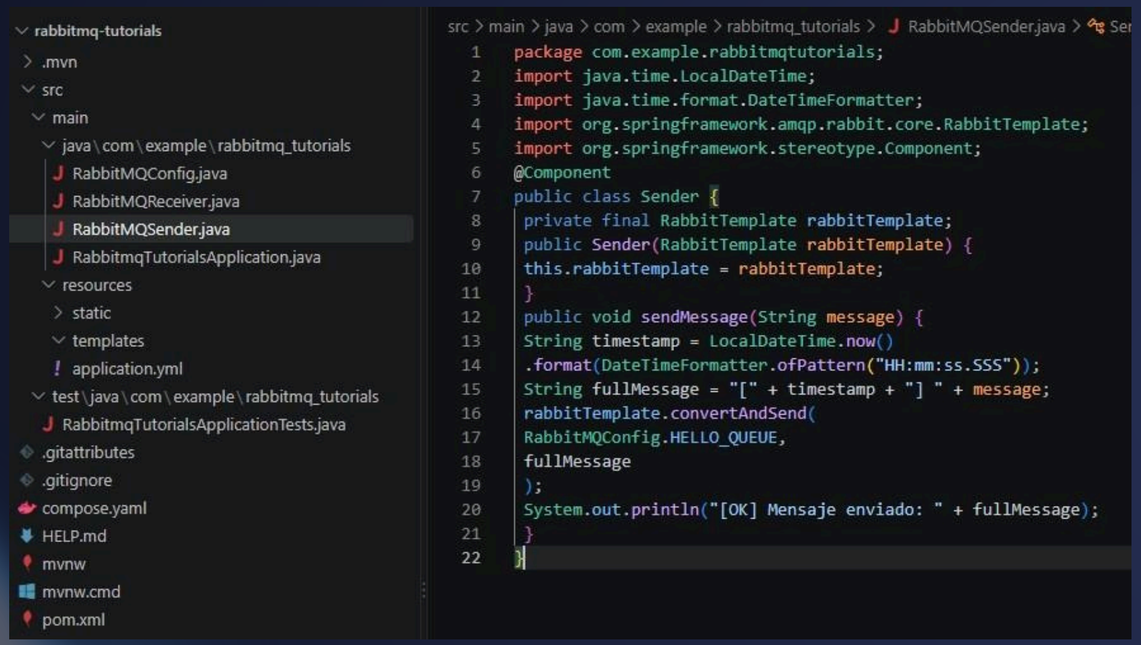
RabbitMQ se ejecutó en un contenedor Docker utilizando una definición Compose. Se publicaron los puertos 5672 para AMQP y 15672 para la interfaz web de administración.



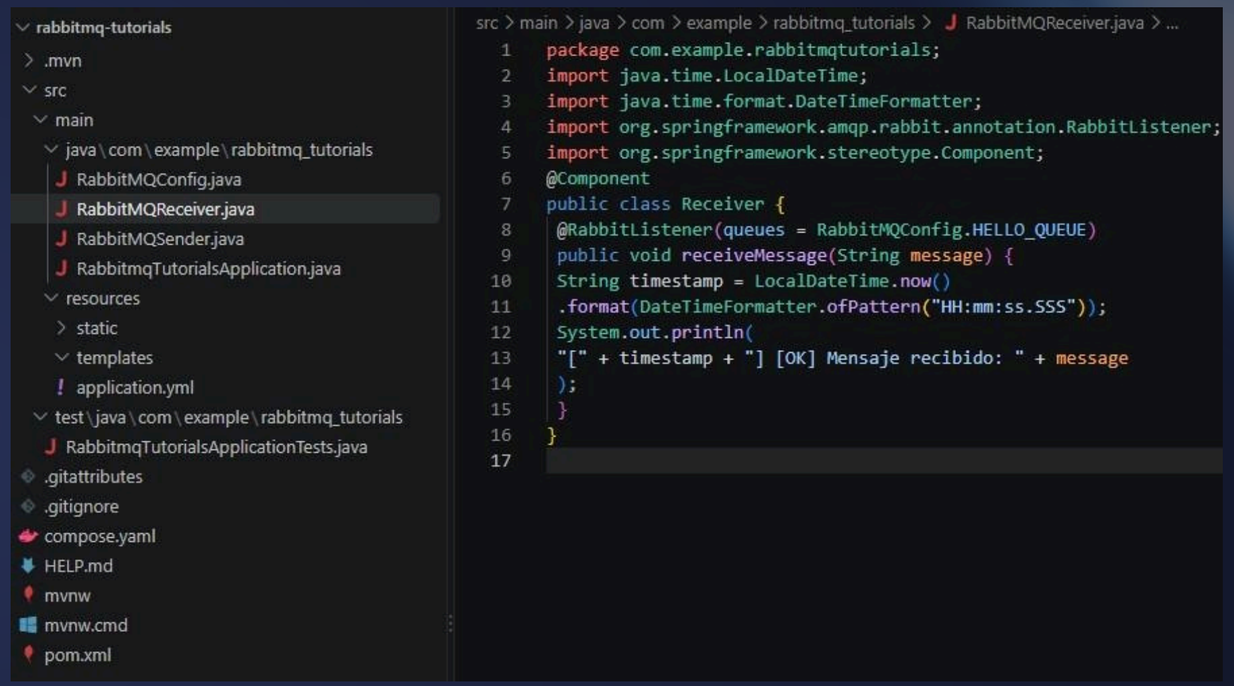
Primera actividad

Cola, productor y consumidor

1. **Generación del proyecto Spring Boot**
2. **Configuración de conexión en application.yml**
3. **Implementación del productor Sender**



4. **Implementación del consumidor Receiver**



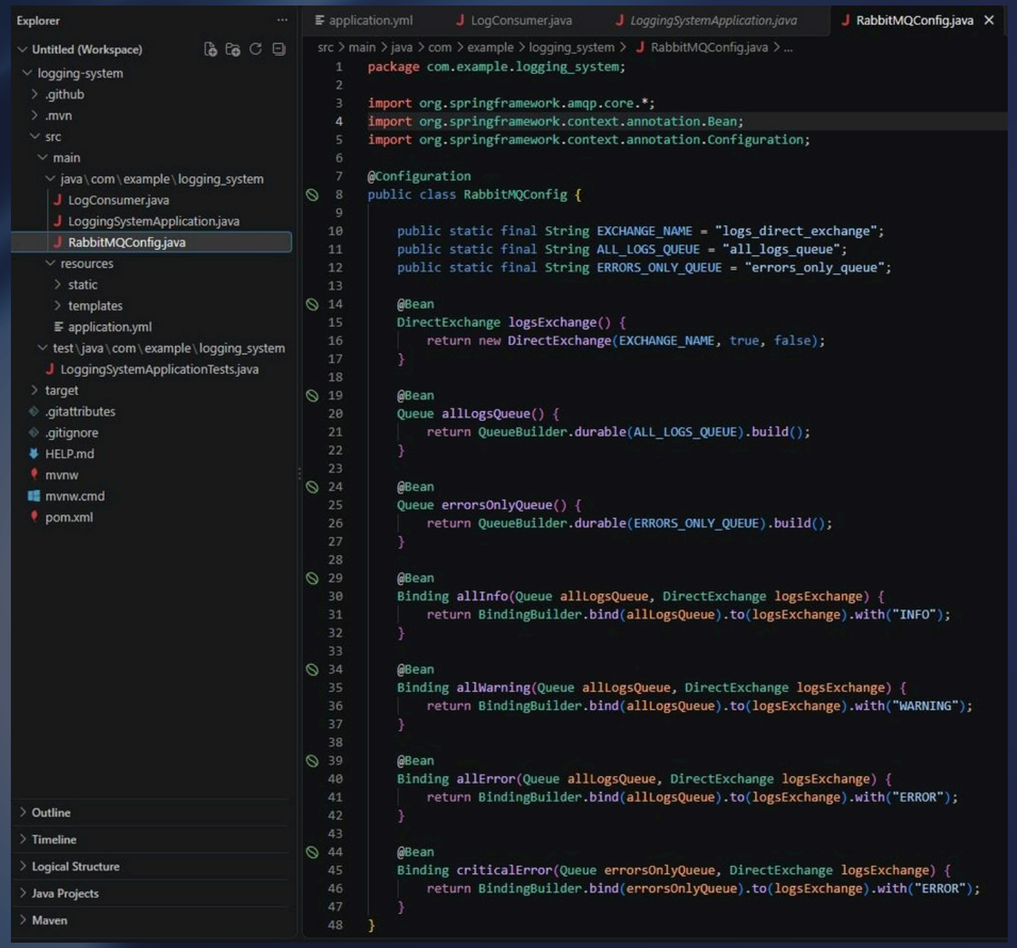
5. **Ejecución de la primera prueba de envío y recepción de mensaje**

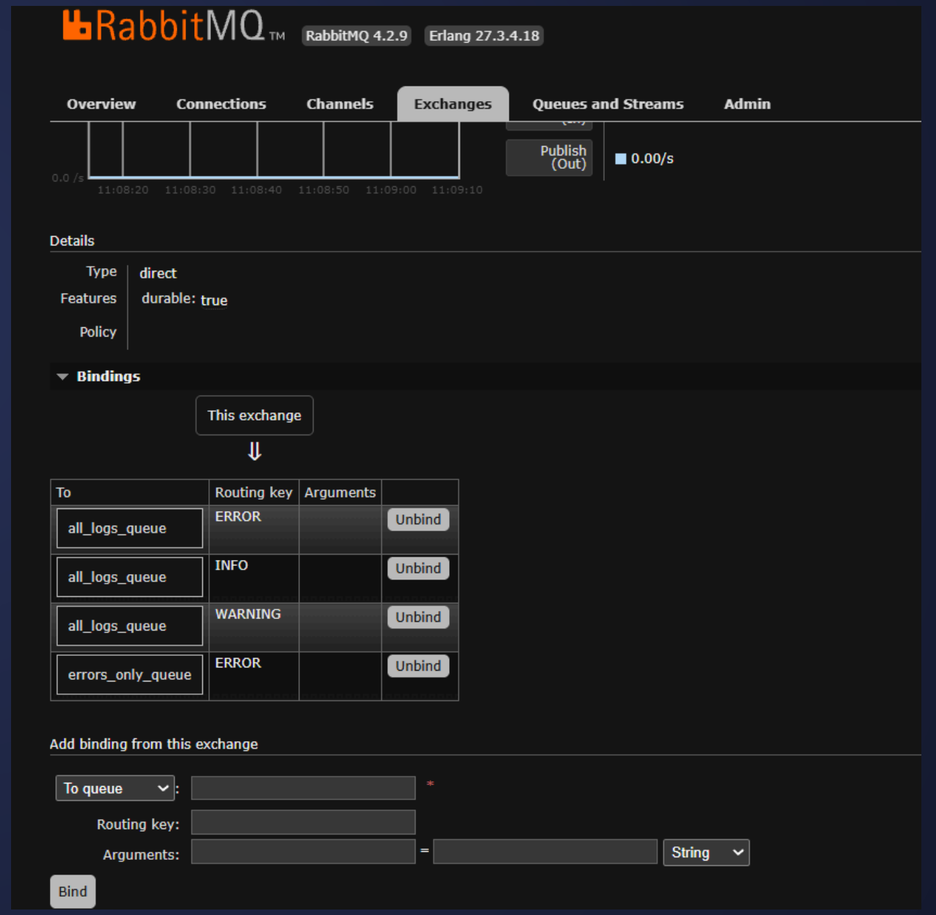
```
1. Enviar mensaje
2. Salir
Seleccione: 1
Mensaje: Hola RabbitMQ
[OK] Mensaje enviado: [15:30:12.512] Hola RabbitMQ

1. Enviar mensaje
2. Salir
Seleccione: [15:30:12.523] [OK] Mensaje recibido: [15:30:12.512] Hola RabbitMQ
```

Segunda actividad

exchange, bindings y routing keys

Se amplió la topología con logs_direct_exchange, all_logs_queue y errors_only_queue. El exchange es de tipo direct, por lo que compara la routing key del mensaje con las binding keys configuradas.

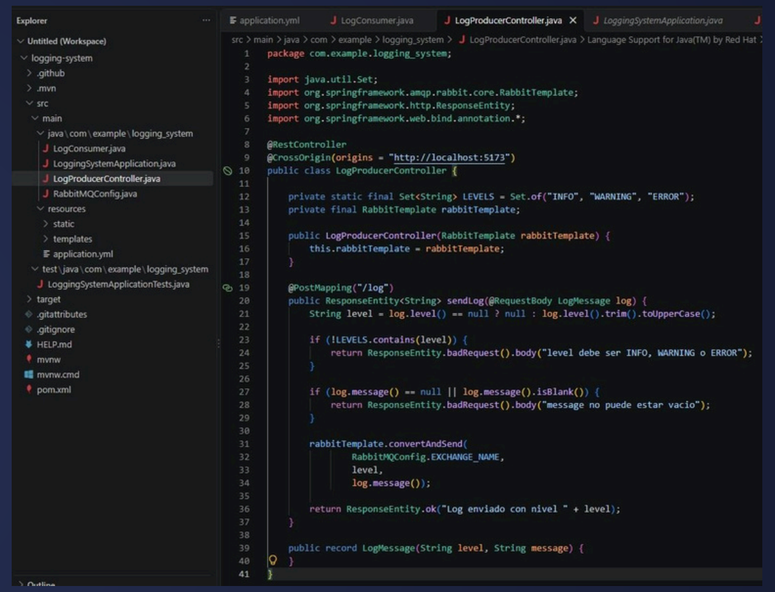


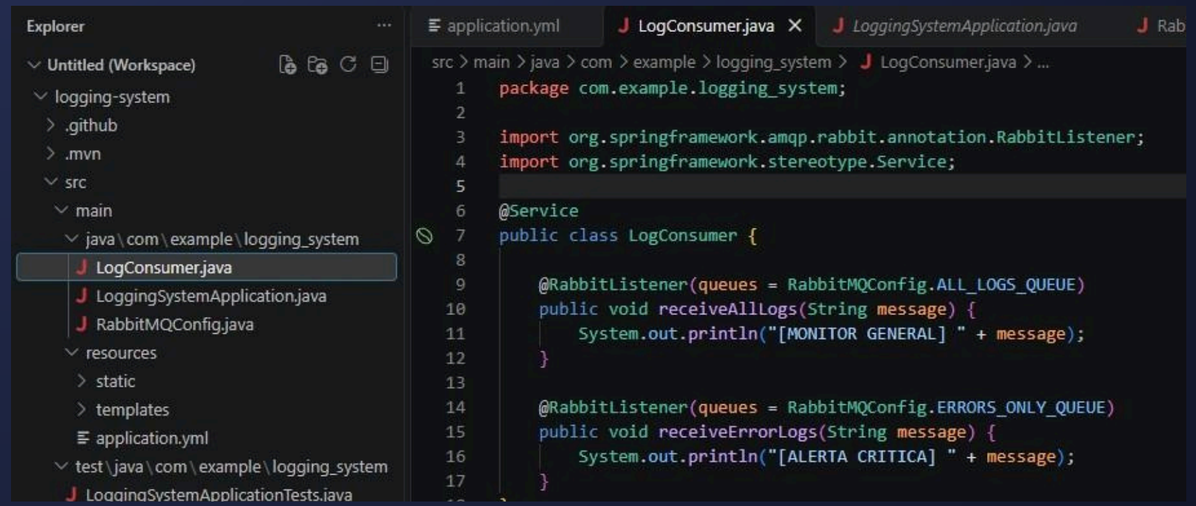


Productor REST y consumidores especializados

Controlador productor que publica mensajes mediante RabbitTemplate

Consumidor de logs configurado con @RabbitListener





Pruebas INFO, WARNING y ERROR

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\basti>curl -X POST http://localhost:8080/log -H "Content-Type: application/json" -d "{\"level\": \"INFO\", \"message\": \"Usuario inicio sesion\"}"
Log enviado con nivel INFO
C:\Users\basti>
```

```
-X POST http://localhost:8080/log -H "Content-Type: application/json" -d "{\"level\": \"WARNING\", \"message\"
l WARNING
```

```
C:\Users\basti>curl -X POST http://localhost:8080/log -H "Content-Type: application/json" -d "{\"level\": \"ERROR\", \"message\": \"Fallo de base de datos\"}"
Log enviado con nivel ERROR
C:\Users\basti>
```

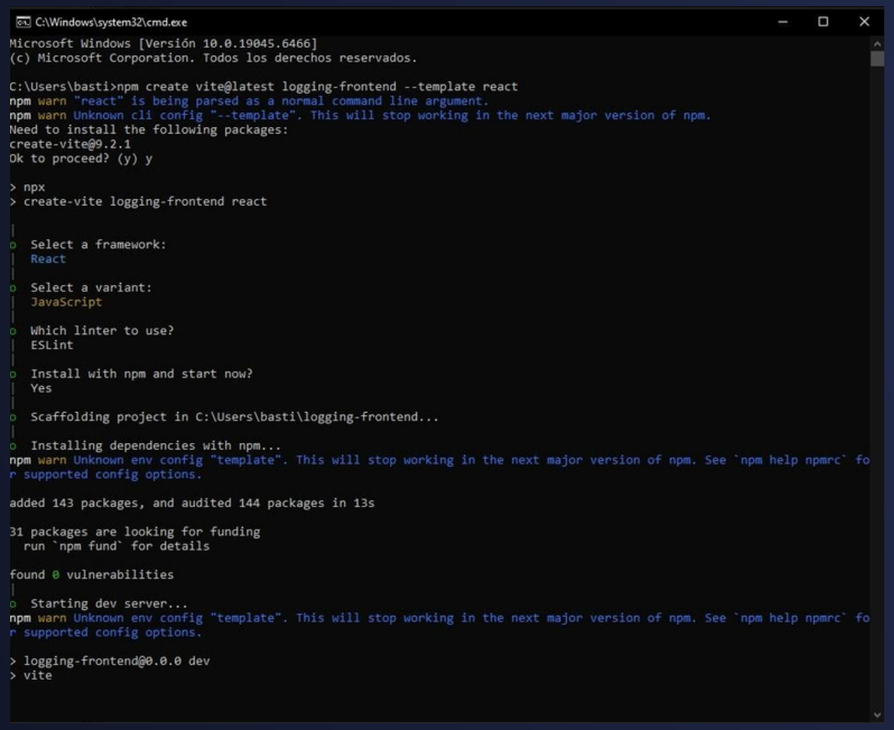
```
[MONITOR GENERAL] Usuario inicio sesion
[MONITOR GENERAL] CPU al 85%
[MONITOR GENERAL] Fallo de base de datos
[ALERTA CRITICA] Fallo de base de datos
```

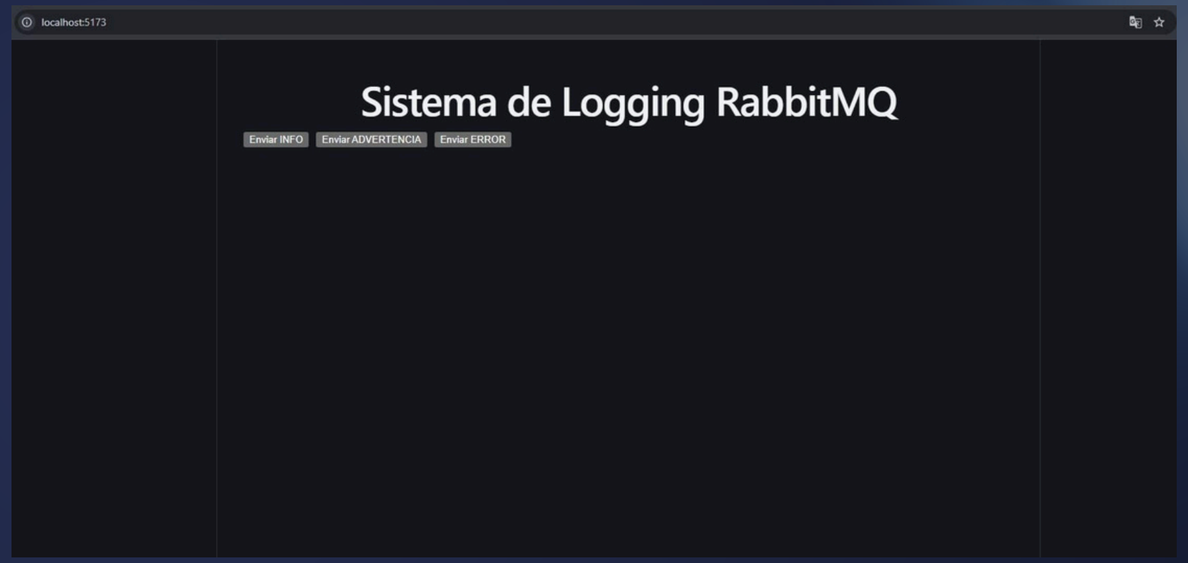
Resultados:

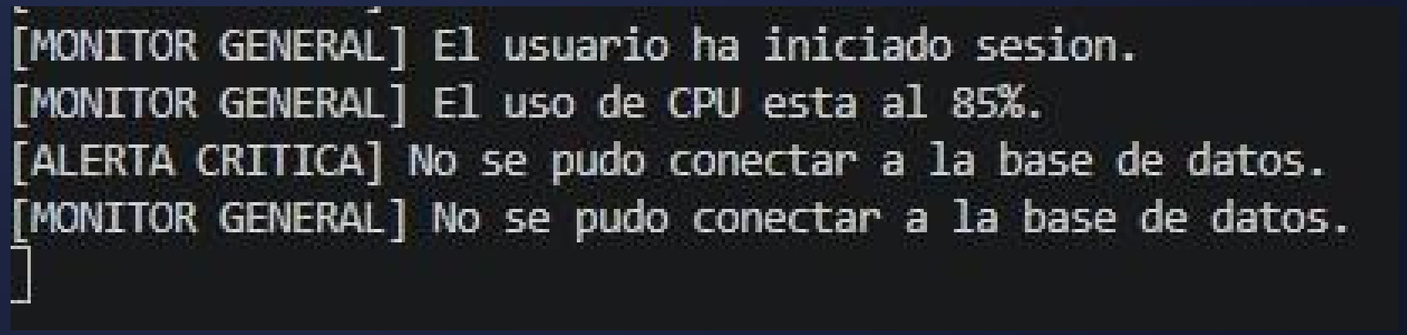
Routing key	Monitor general	Alerta crítica
INFO	Sí	No
WARNING	Sí	No
ERROR	Sí	Sí

Integración con frontend React/Vite

Posteriormente se creó un frontend React/Vite para reemplazar las pruebas manuales por una interfaz de usuario. El formulario permite seleccionar INFO, WARNING o ERROR, escribir un mensaje y enviarlo mediante HTTP POST al backend







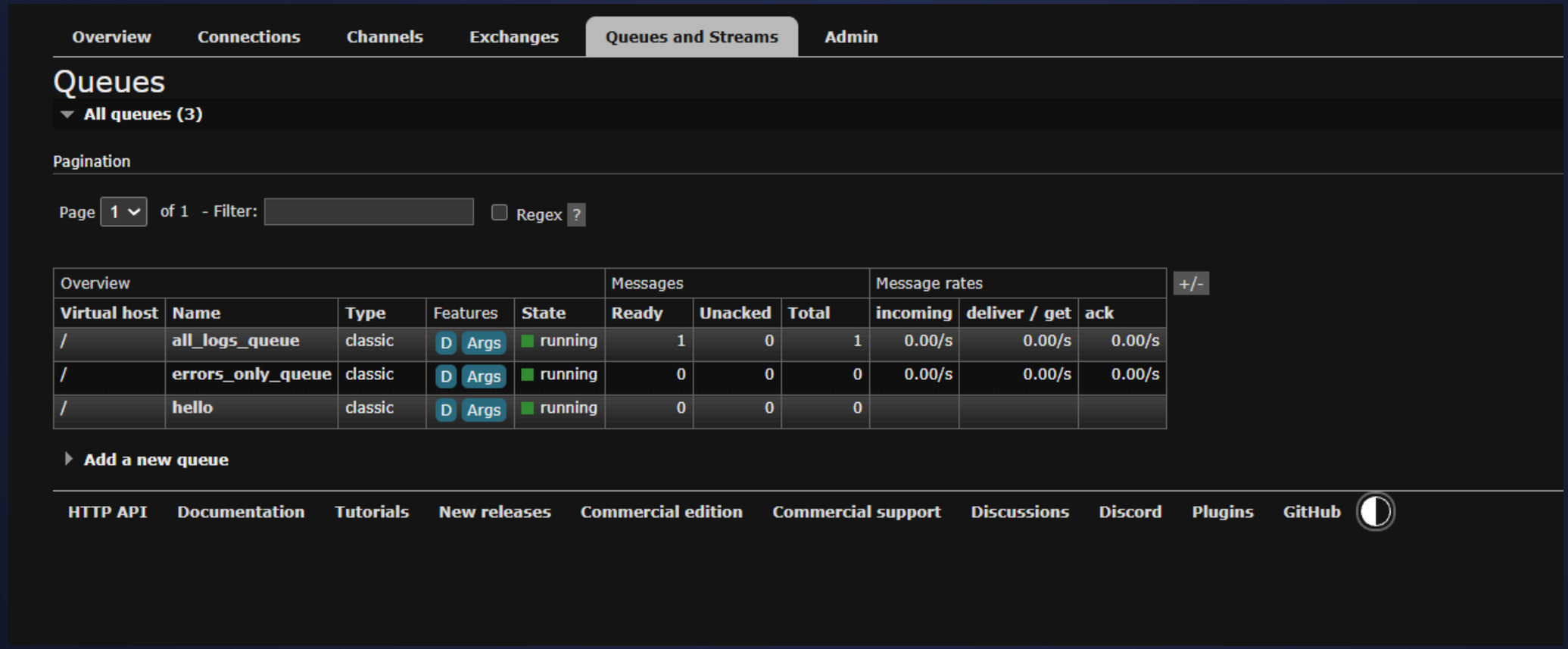
Manejo de errores y recuperación

El frontend incorpora manejo de errores mediante try/catch. Para comprobarlo se detuvo Spring Boot de forma intencional y se intentó enviar un mensaje desde React.

Con el backend detenido, el frontend informa que el registro no pudo enviarse y conserva el mensaje para permitir un reintento

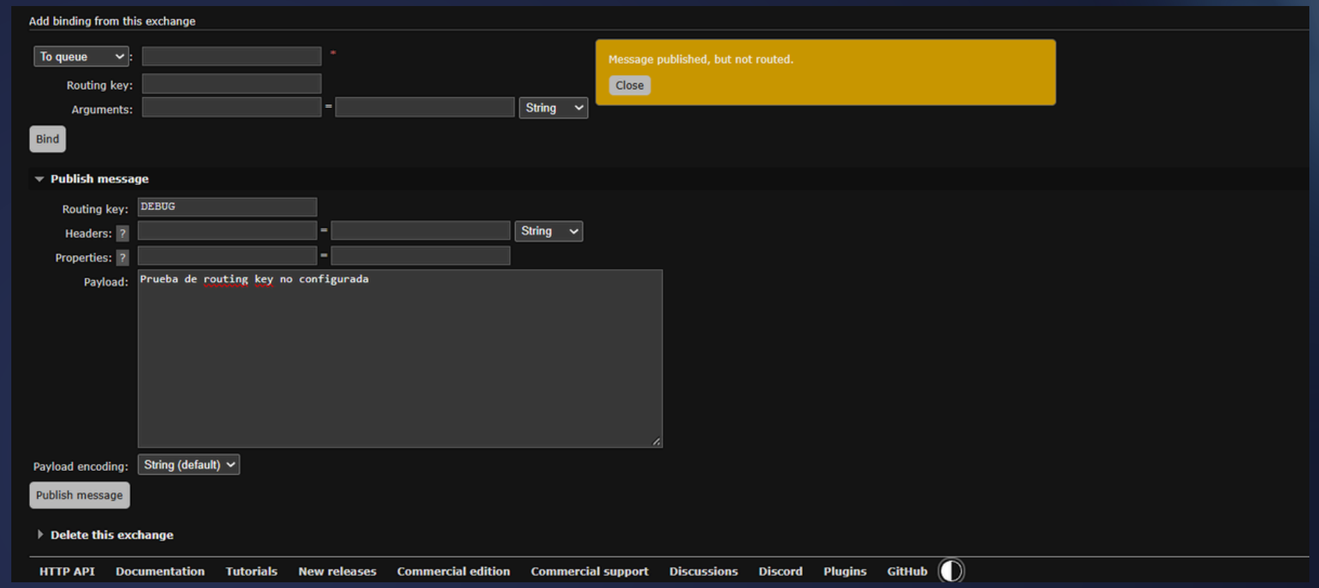
Persistencia temporal de mensajes

Para demostrar el desacoplamiento temporal, Spring Boot fue detenido y se publicó un mensaje INFO directamente desde RabbitMQ Management. Como no existía un consumidor activo, el mensaje permaneció almacenado en la cola

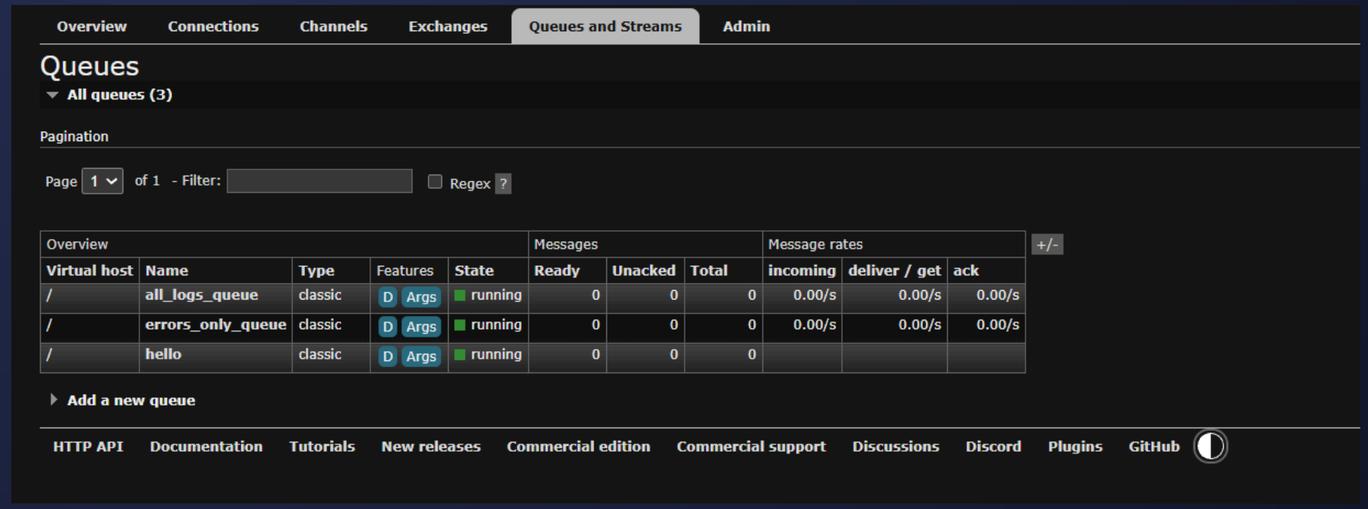


Routing key no configurada

Como prueba final de enrutamiento se utilizó la routing key DEBUG, que no posee ningún binding en logs_direct_exchange. RabbitMQ recibió la publicación, pero no encontró una cola de destino.



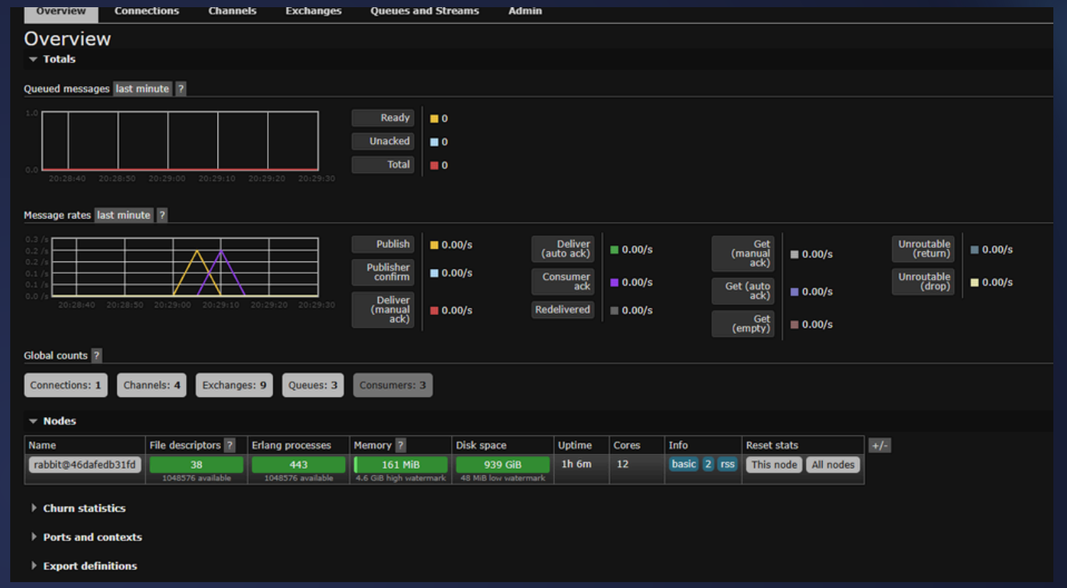
La prueba confirma el comportamiento esperado de un DirectExchange: la coincidencia entre routing key y binding key debe ser exacta

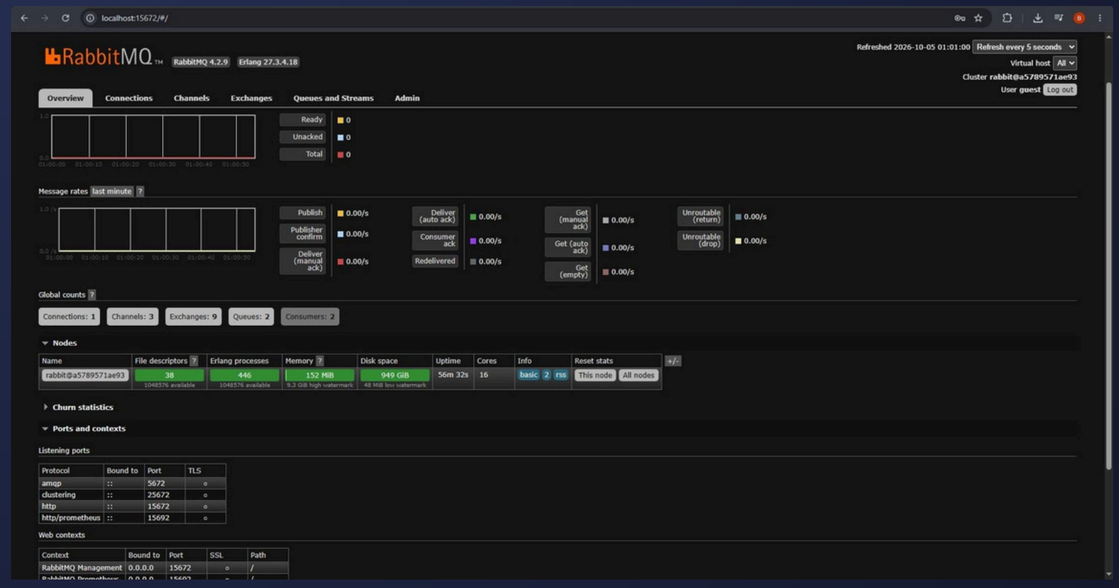


Evidencias finales en RabbitMQ Management

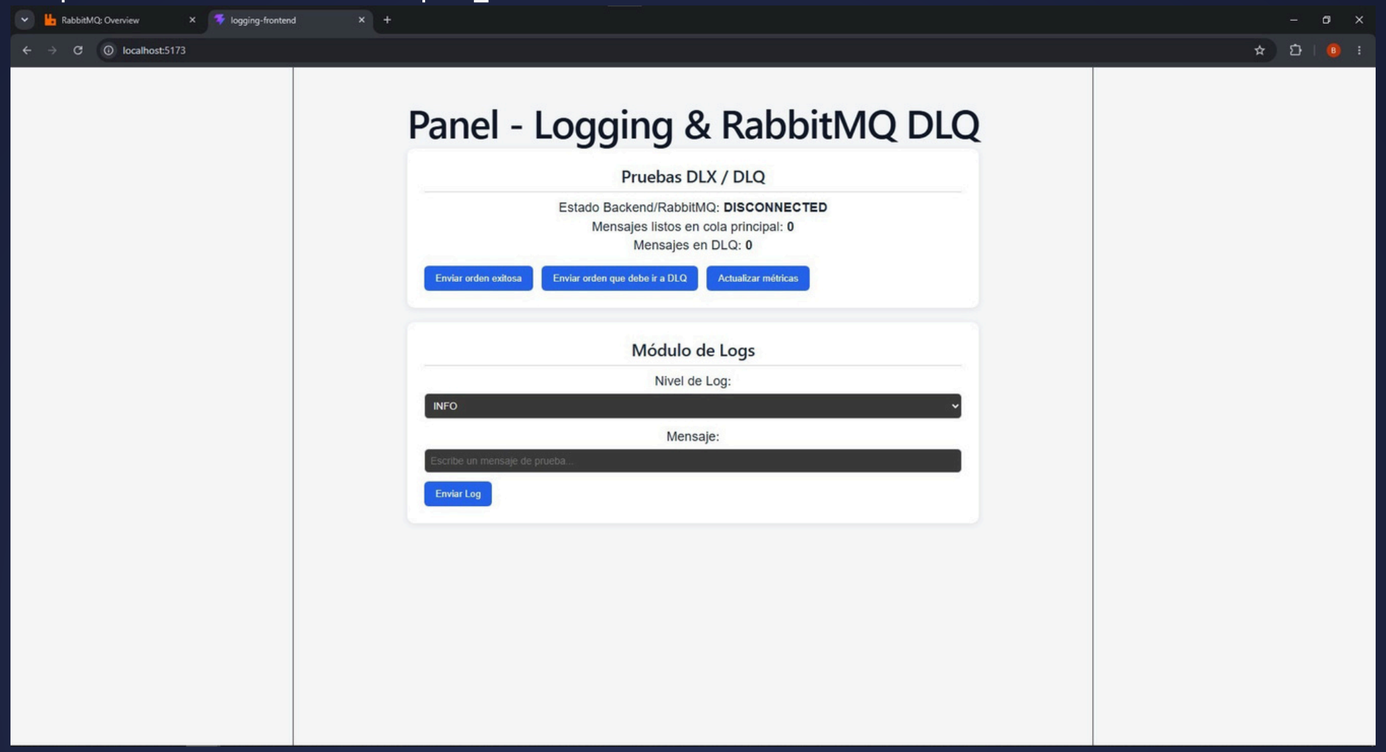
La interfaz de RabbitMQ permitió comprobar el estado general de la solución una vez integrados backend, frontend y consumidores.

Estas evidencias complementan las salidas de Spring Boot: mientras la terminal demuestra el procesamiento de los consumidores, RabbitMQ Management permite inspeccionar la topología y el estado del broker

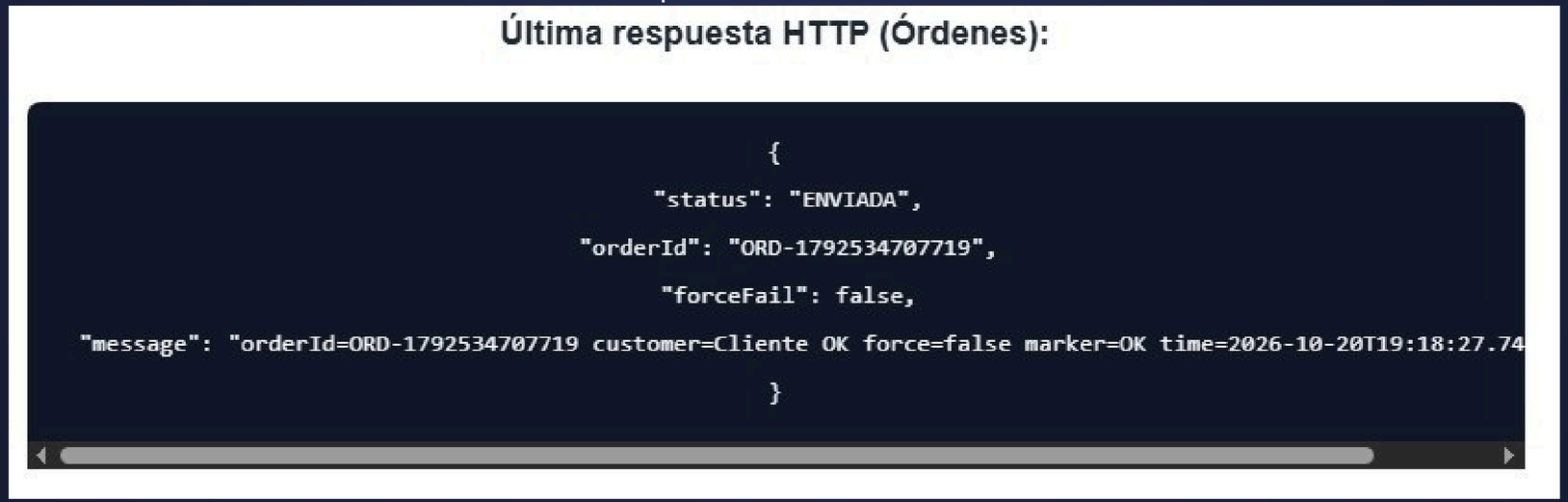




Captura del frontend desplegado



Confirmación del frontend al enviar la respuesta exitosa



Respuesta del backend a la respuesta exitosa --->

```
Problems 2  Output  Debug Console  Terminal

[INFO]
[INFO] --- spring-boot:4.0.8:run (default-cli) @ logging-system ---
[INFO] Attaching agents: []

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v4.0.8)

2026-10-20T19:18:03.928-03:00  INFO 277092 --- [logging-system] [           main] c.e.l.LoggingSystemApplication           : Starting LoggingSystemApplication using Java 21.0.10
logging-system\target\classes started by basti in C:\Users\basti\Desktop\logging-system)
2026-10-20T19:18:03.931-03:00  INFO 277092 --- [logging-system] [           main] c.e.l.LoggingSystemApplication           : No active profile set, falling back to 1 default prof
2026-10-20T19:18:04.645-03:00  INFO 277092 --- [logging-system] [           main] o.s.boot.tomcat.TomcatWebServer          : Tomcat initialized with port 8080 (http)
2026-10-20T19:18:04.659-03:00  INFO 277092 --- [logging-system] [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-10-20T19:18:04.659-03:00  INFO 277092 --- [logging-system] [           main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/11.0.24]
2026-10-20T19:18:04.705-03:00  INFO 277092 --- [logging-system] [           main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed
2026-10-20T19:18:05.118-03:00  INFO 277092 --- [logging-system] [           main] o.s.boot.tomcat.TomcatWebServer          : Tomcat started on port 8080 (http) with context path
2026-10-20T19:18:05.120-03:00  INFO 277092 --- [logging-system] [           main] o.s.a.r.c.CachingConnectionFactory       : Attempting to connect to: [localhost:5672]
2026-10-20T19:18:05.161-03:00  INFO 277092 --- [logging-system] [           main] o.s.a.r.c.CachingConnectionFactory       : Created new connection: rabbitConnectionFactory#2db2a
e=amqp://guest@127.0.0.1:5672/, localPort=55391]
2026-10-20T19:18:05.248-03:00  INFO 277092 --- [logging-system] [           main] c.e.l.LoggingSystemApplication           : Started LoggingSystemApplication in 1.664 seconds (pr
2026-10-20T19:18:06.841-03:00  INFO 277092 --- [logging-system] [nio-8080-exec-2] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServ
2026-10-20T19:18:06.842-03:00  INFO 277092 --- [logging-system] [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
2026-10-20T19:18:06.843-03:00  INFO 277092 --- [logging-system] [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet        : Completed initialization in 1 ms
[PROCESADOR] Recibida: orderId=ORD-1792534707719 customer=Cliente OK force=false marker=OK time=2026-10-20T19:18:27.746487200
[ÉXITO] Orden procesada correctamente.
```

Respuesta del frontend al enviar una orden fallida a dlq --->

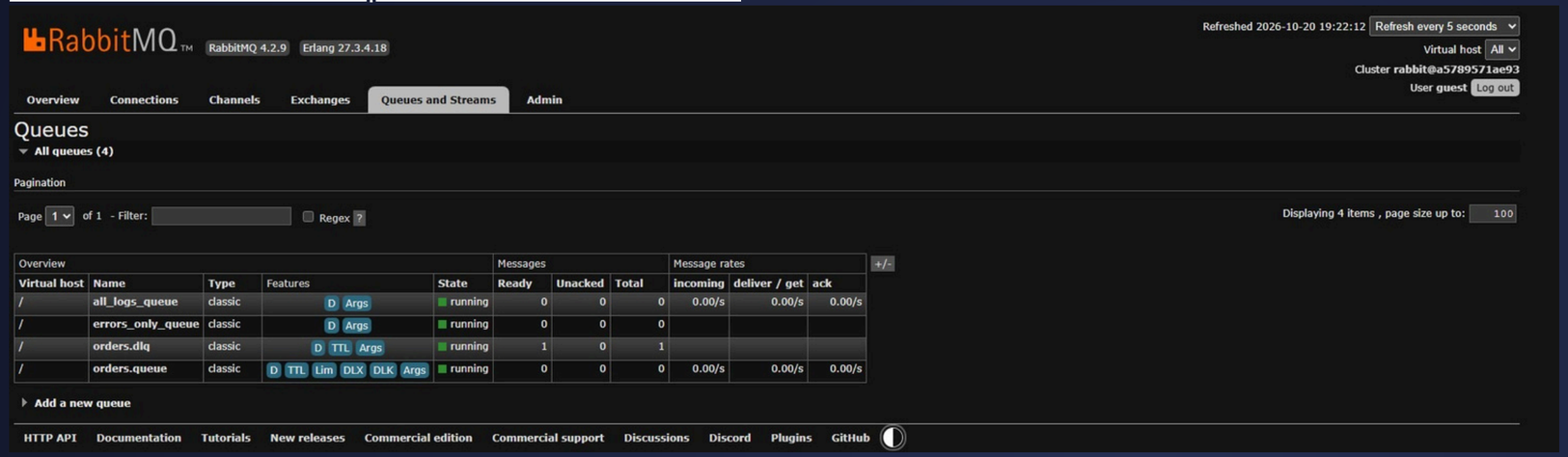

```
{
  "status": "ENVIADA",
  "orderId": "ORD-1792534795047",
  "forceFail": true,
  "message": "orderId=ORD-1792534795047 customer=Cliente DLQ force=true marker=FAIL time=2026-10-20T19:19:55.
}
```


Respuesta del backend sobre la orden fallida a dlq -->

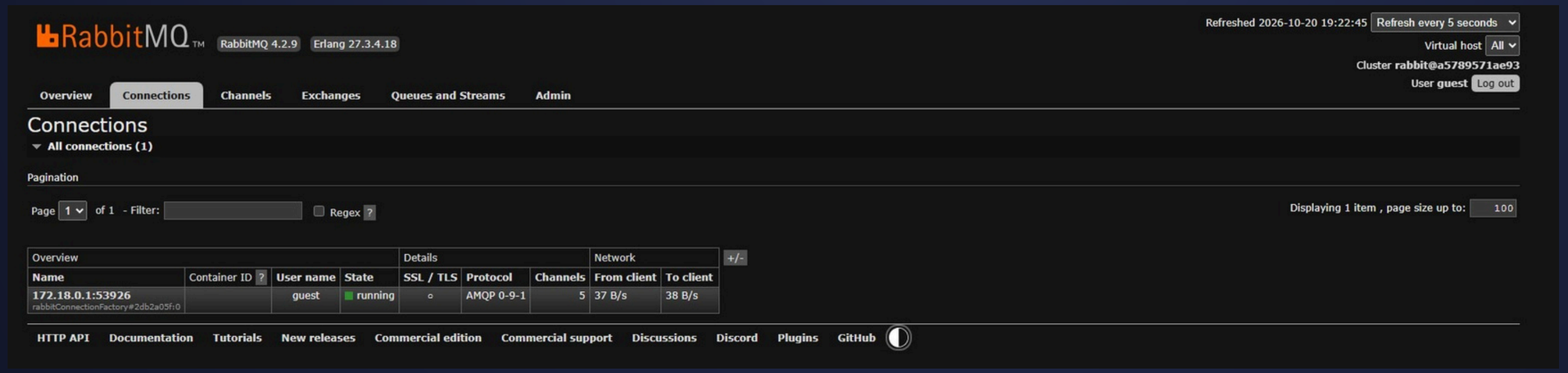
```
[PROCESADOR] Recibida: orderId=ORD-1792534795047 customer=Cliente DLQ force=true marker=FAIL time=2026-10-20T19:19:55.048524700
[ERROR SIMULADO] La orden fue marcada para fallar.
[PROCESADOR] Recibida: orderId=ORD-1792534795047 customer=Cliente DLQ force=true marker=FAIL time=2026-10-20T19:19:55.048524700
[ERROR SIMULADO] La orden fue marcada para fallar.
[PROCESADOR] Recibida: orderId=ORD-1792534795047 customer=Cliente DLQ force=true marker=FAIL time=2026-10-20T19:19:55.048524700
[ERROR SIMULADO] La orden fue marcada para fallar.
[PROCESADOR] Recibida: orderId=ORD-1792534795047 customer=Cliente DLQ force=true marker=FAIL time=2026-10-20T19:19:55.048524700
[ERROR SIMULADO] La orden fue marcada para fallar.
```

Estado de colas activas en la consola de RabbitMQ

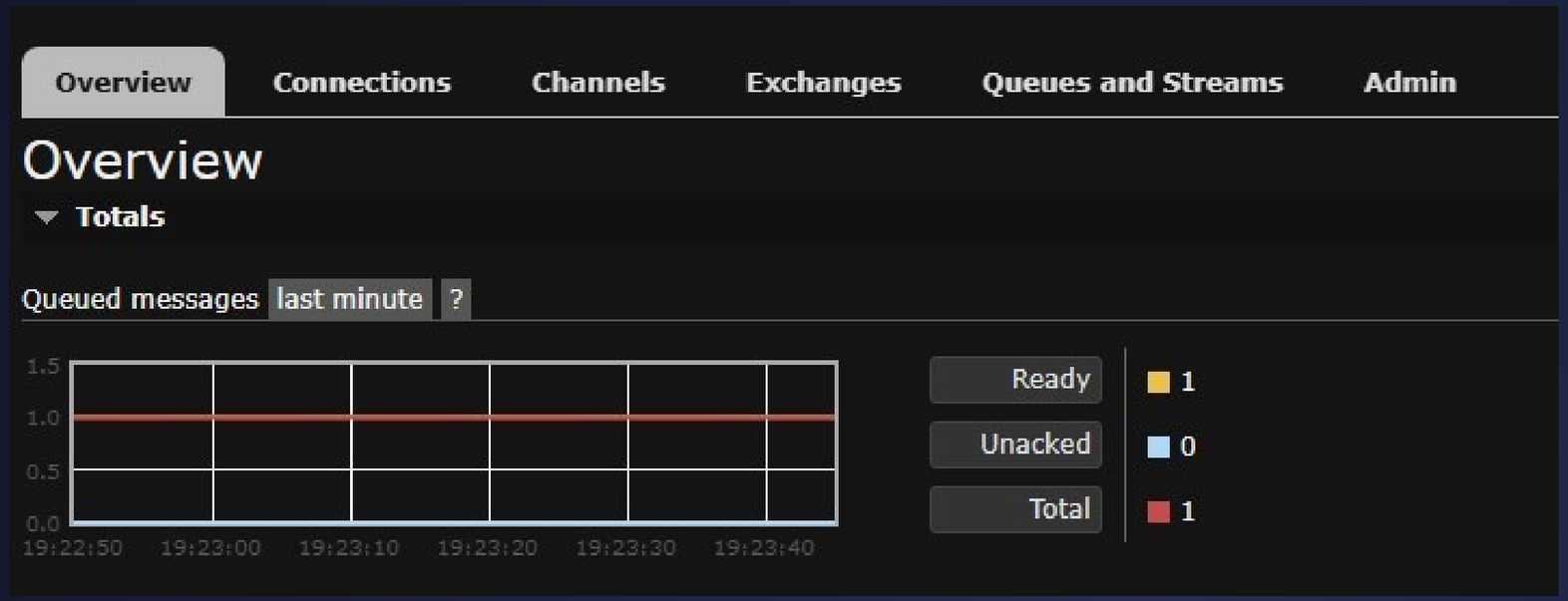
Evidencias de rabbitmq sobre las colas activas



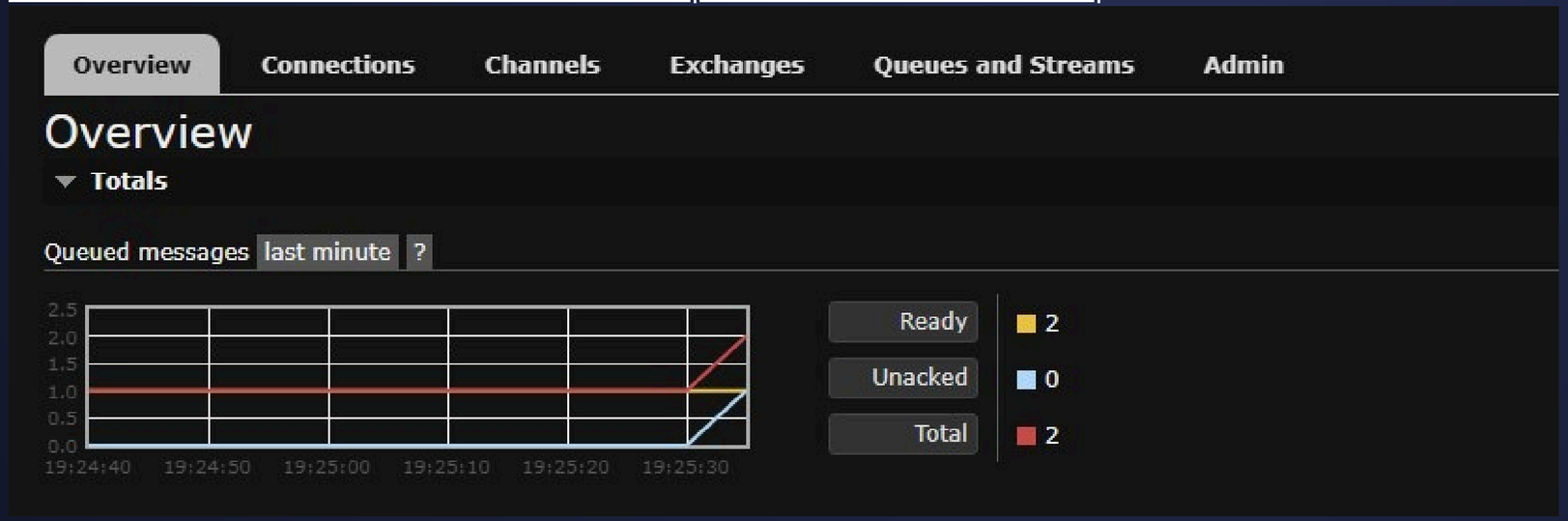
Evidencias de la conexión de rabbitmq



Evidencias de las colas de rabbitmq



Más evidencias de las métricas de rabbitmq al enviar otra orden a dlq



Conclusión